

Aufgabe 4: Cache-Kohärenz (Ring-Modell)

Gegeben ist ein 4-Core-System, bei dem Cache-Änderungen über einen Ring weitergegeben werden. Pro passierendem Kern wird ein Takt benötigt, und jeder empfangende Kern leitet die Änderung an anliegende Kerne weiter. Es wird jeweils der „aktuellere“ Wert übernommen.

Erfüllt das Modell sequentielle Kohärenz?

Nein, nicht in allen Fällen.

Gegenbeispiel read own writes:

Ein älteres Datum wird nach store zu Core 0 propagiert:

- t : core 3 sets $x = 3$
- $t+1$: core 0 sets $x = 0$
- $t+2$: core 0 gets propagated value $x = 3$ from 3's write at t
- $t+3$: core 0 reads $x = 3$

Gegenbeispiel write serialization:

- t : core 0 sets $x=0$
- $t+1$: core 2 sets $x=1$; core 0 reads $x=0$
- $t+2$: core 2 reads $x=1$
- $t+3$: $x=0$ propagates to core 2
- $t+4$: core 2 reads $x=0$
- $t+5$: $x=0$ propagates to core 0; core 0 reads $x=0$

Hier 'sieht' Core 2 die Sequenz 1, 0, Core 0 hingegen 0, 1.

Was muss man ändern?

Man benötigt eine **globale Serialisierung** von Writes (total order), damit alle Kerne dieselbe Reihenfolge der Schreiboperationen sehen.

Eine einfache Lösung ist ein **Token**-Mechanismus: Nur der Kern, der aktuell das Token besitzt, darf schreiben. Nach einer Schreiboperation wird das Token weitergereicht. Damit werden alle Writes automatisch in eine eindeutige Reihenfolge gebracht.

Alternative: Jede Schreiboperation bekommt eine globale Sequenznummer (z.B. durch das Token oder einen zentralen Zähler). Updates werden von allen Kernen strikt nach dieser Nummer angewendet (ggf. mit Puffern/Warten), sodass alle Kerne dieselbe Reihenfolge sehen.

Damit wird insbesondere **Write serialization** eingehalten und das Modell kann sequentielle Kohärenz erfüllen.